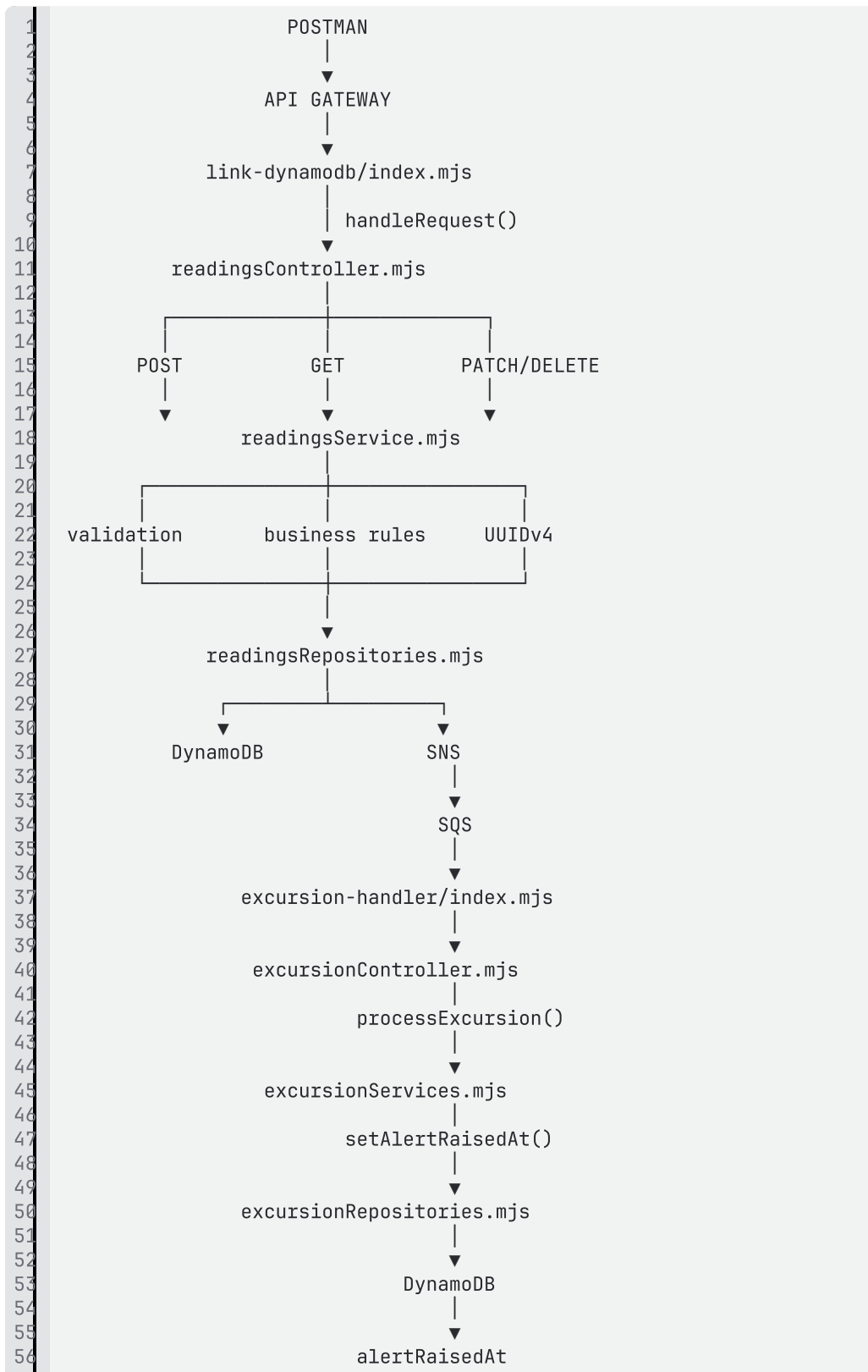


Understanding the complete connection between every JS file

This is probably the diagram I want you to remember most:



Follow one POST through every file

This is a really good way to understand it.

Postman sends:

```
1 {  
2   "recordedAt": "2026-08-31T14:00:00.000Z",  
3   "minTempC": 3,  
4   "maxTempC": 12,  
5   "recordedBy": "Umar"  
6 }
```

Step 1

```
1 API Gateway
```

recognises:

```
1 POST /branches/{branchId}/readings
```

and invokes the main Lambda.

Step 2

```
1 index.mjs
```

calls:

```
1 handleRequest(  
2   event,  
3   context  
4 );
```

Step 3

```
1 readingsController.mjs
```

recognises:

```
1 POST
```

parses the JSON and calls:

```
1 createReading(  
2   branchId,  
3   body  
4 );
```

Step 4

```
1 readingsService.mjs
```

validates:

```
1 recordedAt  
2 minTempC  
3 maxTempC
```

```
4 recordedBy
```

generates:

```
1 UUIDv4
```

then calculates:

```
1 maxTempC = 12
2   ↓
3 status = excursion
```

Step 5

Service calls:

```
1 saveReading(item);
```

Step 6

```
1 readingsRepositories.mjs
```

uses:

```
1 PutCommand
```

and saves to DynamoDB.

Step 7

Because:

```
1 status = excursion
```

the service calls:

```
1 publishExcursion(...)
```

Step 8

Repository uses:

```
1 PublishCommand
```

and publishes to SNS.

Step 9

SNS delivers to:

```
1 SQS
```

Step 10

SQS invokes:

```
1 excursion-handler/index.mjs
```

Step 11

Index passes the SQS event to:

```
1 | excursionController.mjs
```

Step 12

Controller parses:

```
1 | record.body
```

and calls:

```
1 | processExcursion(message);
```

Step 13

```
1 | excursionServices.mjs
```

creates:

```
1 | alertRaisedAt
```

and calls:

```
1 | setAlertRaisedAt(...)
```

Step 14

```
1 | excursionRepositories.mjs
```

uses:

```
1 | UpdateCommand
```

to update DynamoDB.

Final item:

```
1 | readingId
2 | branchId
3 | recordedAt
4 | minTempC
5 | maxTempC
6 | recordedBy
7 | status = excursion
8 | alertRaisedAt
```

The easiest debugging rule

This will save you a lot of time.

If something goes wrong with:

```
1 | wrong API route
```

```
2 wrong HTTP response
3 JSON body parsing
```

look at:

```
1 readingsController.mjs
```

If something goes wrong with:

```
1 temperature validation
2 UUID generation/validation
3 status calculation
4 PATCH rules
5 future timestamp
```

look at:

```
1 readingsService.mjs
```

If something goes wrong with:

```
1 DynamoDB Get
2 DynamoDB Put
3 DynamoDB Query
4 DynamoDB Update
5 DynamoDB Delete
6 SNS Publish
```

look at:

```
1 readingsRepositories.mjs
```

If:

```
1 SQS message isn't being interpreted correctly
```

look at:

```
1 excursionController.mjs
```

If:

```
1 alert rules
2 forceFailure
3 alertRaisedAt logic
```

look at:

```
1 excursionServices.mjs
```

If:

```
1 DynamoDB isn't setting alertRaisedAt
2 duplicate message protection isn't working
```

look at:

```
1 excursionRepositories.mjs
```

And if Lambda doesn't seem to start at all:

```
1 | index.mjs
```

is the first JavaScript place to check.

The simple mental model is:

```
1 | INDEX
2 | "Start the Lambda."
3 |
4 |   ↓
5 |
6 | CONTROLLER
7 | "What was requested?"
8 |
9 |   ↓
10 |
11 | SERVICE
12 | "Is it allowed, and what should happen?"
13 |
14 |   ↓
15 |
16 | REPOSITORY
17 | "How do I communicate with AWS?"
```

That is the relationship between **every JavaScript file in your current project**.